Метод опорных векторов. Часть 1

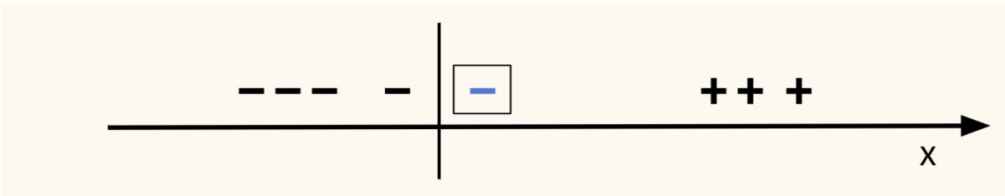
Рассмотрим **метод опорных векторов** (support vector machine, SVM).

SVM с жестким зазором

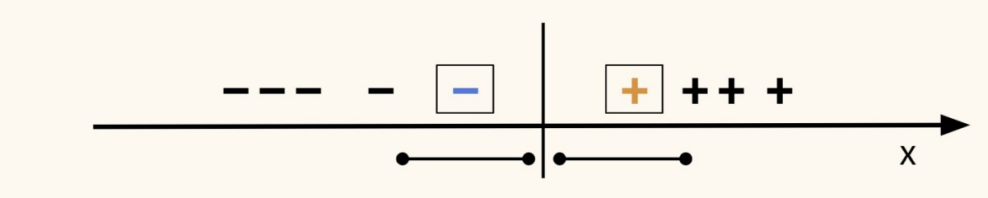
Основная идея

Пусть дана задача классификации с единственным признаком x и двумя классами, которые мы обозначим как $-$ и $+$. **Решающая граница** (decision boundary) позволит отнести новый объект к одному из двух классов.

При этом свободное расположение границы может привести к ошибочной классификации. В частности, на рисунке ниже новое наблюдение находится намного ближе к классу $-$, но классифицируется как $+$.



Представляется разумным расположить границу таким образом, чтобы максимизировать *зазор* или *отступ* (margin) между крайними или *опорными* наблюдениями и самой этой границей.



Очевидно, что тогда граница будет равноудалена от опорных объектов.

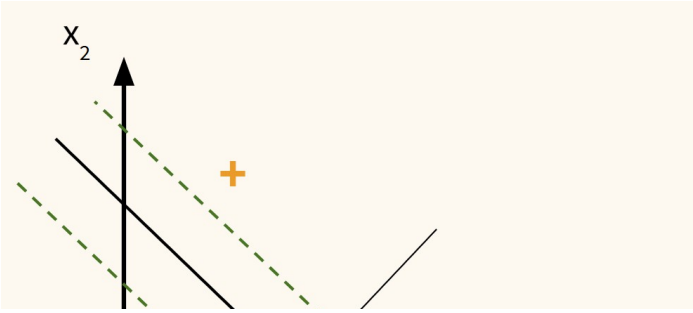
> Такой алгоритм называется методом **классификатора с максимальным зазором** (maximal margin classifier) или **методом опорных векторов с жестким зазором** (hard-margin SVM), поскольку *не допускает* ошибочной классификации в процессе обучения.

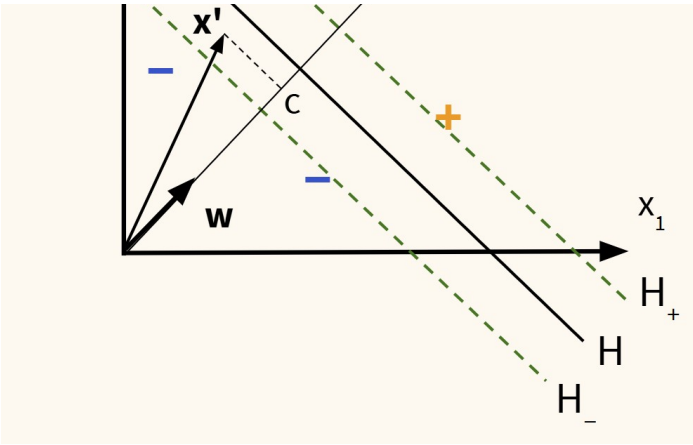
Примечание. О том, зачем позволять алгоритму ошибаться, мы поговорим чуть ниже.

Несложно заметить, что этот алгоритм подходит для линейно разделимых данных. Посмотрим, каким образом можно построить решающую границу, а точнее, в случае SVM, **разделяющую гиперплоскость** (hyperplane).

Разделяющая гиперплоскость

Рассмотрим набор данных $\mathcal{D} = \{\mathbf{x}_i, y_i\}_{i=1}^N$, в котором $\mathbf{x}_i \in \mathbb{R}^2$ и $y \in \{-1, +1\}$. Построим разделяющую гиперплоскость H, а также границы коридора (или отрицательные и положительные гиперплоскости) H_- и H_+.





Пусть вектор $\mathbf{w}$ произвольной длины ортогонален H. Рассмотрим объект $\mathbf{x}'$ и попробуем определить, находится ли он по «правую» или по «левую» сторону от H. Зададим правило

$$\mathbf{w}^\top \mathbf{x} \geq c,$$

согласно которому в зависимости от величины проекции c на вектор $\mathbf{w}$ мы окажемся на «левой» или «правой» стороне от гиперплоскости H в направлении $\mathbf{w}$. Положим $c = -b$. Тогда,

- если $\mathbf{w}^\top \mathbf{x} + b \geq 0$, то объект находится «слева» от H и принадлежит к классу $+$;
- если же $\mathbf{w}^\top \mathbf{x} + b < 0$, то «справа» от H и принадлежит к классу $-$.

Таким образом,

$$H = \{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}$$

Введем (произвольное) правило для наблюдений, согласно которому объекты будут отнесены к классу $+$, если

$$\mathbf{w}^\top \mathbf{x}_+ + b \geq 1$$

и к классу $-$, если

$$\mathbf{w}^\top \mathbf{x}_- + b \leq -1$$

Если положить y_i, такое что

$$y_i = \begin{cases} +1 & \text{для класса } +; \\ -1 & \text{для класса } -, \end{cases}$$

то решающее правило для гиперплоскости можно свести к

$$y_i \left[(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \right]$$

с учетом замены знака неравенства на противоположный при умножении на $y_i = -1$.

При этом границы коридора задаются через

$$H_- = \{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = -1\}$$
$$H_+ = \{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = +1\}$$

Преобразуем решающее правило

$$y_i [(\mathbf{w}^\top \mathbf{x}_i + b) - 1 \geq 0]$$

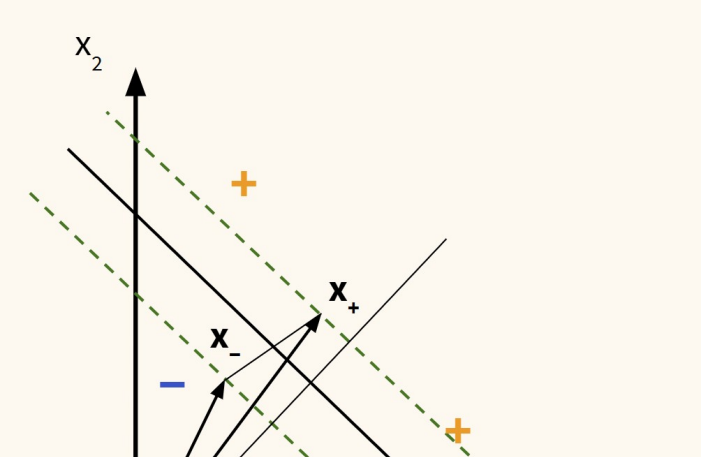
и введем условие

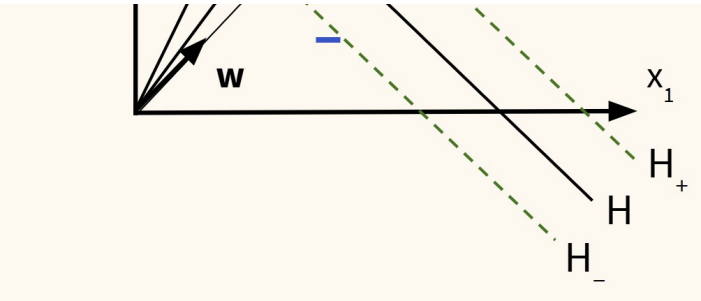
$$y_i [(\mathbf{w}^\top \mathbf{x}_i + b) - 1 = 0]$$

для тех объектов $\mathbf{x}_i$, называемых **опорными векторами** (support vectors), которые лежат на границах коридора.

Ширина коридора

Рассмотрим два вектора $\mathbf{x}_-$ и $\mathbf{x}_+$, лежащих на H_- и H_+ соответственно.





Найдем разницу проекций на $\mathbf{w}$.

$$\text{ширина} = \frac{\mathbf{w}}{||\mathbf{w}||} \cdot \mathbf{x}_+ - \frac{\mathbf{w}}{||\mathbf{w}||} \cdot \mathbf{x}_- = \frac{(\mathbf{w} \cdot \mathbf{x}_+ - \mathbf{w} \cdot \mathbf{x}_-)}{||\mathbf{w}||}$$

По условию для объектов, лежащих на H_- и H_+,

$$\begin{aligned} \mathbf{w}^\top \mathbf{x}_+ + b &= 1 \\ \mathbf{w}^\top \mathbf{x}_- + b &= -1 \end{aligned}$$

Отсюда,

$$\text{ширина} = \frac{((1-b) - (-1-b))}{||\mathbf{w}||} = \frac{2}{||\mathbf{w}||}$$

Максимизация зазора

Основная задача

Таким образом, **основная задача** (primal problem) оптимизации сводится к

$$\max \frac{2}{||\mathbf{w}||}$$

Поскольку константы не влияют на расположение экстремума, то

$$\max \frac{1}{||\mathbf{w}||}$$

Преобразуем нахождение максимума в поиск минимума

$$\min ||\mathbf{w}|| = \min \frac{1}{2}||\mathbf{w}||^2 = \min \frac{1}{2}\mathbf{w}^\top \mathbf{w}$$

с ограничением $y_i[(\mathbf{w}^\top \mathbf{x}_i + b) - 1 \geq 0]$

Определим функцию Лагранжа

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\mathbf{w}^\top \mathbf{w} - \sum_{i=1}^{N} \alpha_i [y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1]$$

Поясним, что в данном случае нам необходимо найти минимум относительно параметров $\mathbf{w}, b$ и максимум относительно $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_N)$, то есть седловую точку (saddle point) $\mathcal{L}$.

$$\min_{\mathbf{w}, b} \left(\max_{\boldsymbol{\alpha}; \boldsymbol{\alpha} \geq 0} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) \right)$$

В самом деле, вначале отдельно рассмотрим задачу максимизации

$$\max_{\boldsymbol{\alpha}; \boldsymbol{\alpha} \geq 0} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})$$

и найдем такие $\mathbf{w}, b$, что $y_i(\mathbf{w}^\top \mathbf{x}_i + b) < 1$. В этом случае нам не составит труда сделать $\mathcal{L}$ произвольно большой, подобрав соответствующие значения $\boldsymbol{\alpha}$.

Предположим, мы подобрали такие $\mathbf{w}$ и b, что

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 0{,}5$$

Тогда,

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\mathbf{w}^\top \mathbf{w} - \sum_{i=1}^{N} \alpha_i (0{,}5 - 1) = \frac{1}{2}\mathbf{w}^\top \mathbf{w} - \sum_{i=1}^{N} -0{,}5\alpha_i$$

Пусть заданы три множителя α_i, и каждый равен единице. В этом случае,

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\mathbf{w}^\top \mathbf{w} - (-1{,}5) = \frac{1}{2}\mathbf{w}^\top \mathbf{w} + 1{,}5$$

Если α_i равны двум, то

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\mathbf{w}^\top\mathbf{w} + 3$$

и так далее. Как следствие, мы можем записать задачу максимизации как

$$\max_{\boldsymbol{\alpha};\boldsymbol{\alpha}\geq 0} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \begin{cases} \frac{1}{2}\mathbf{w}^\top\mathbf{w} & \text{если } y_i(\mathbf{w}^\top\mathbf{x}_i + b) \geq 1; \\ +\infty & \text{если } y_i(\mathbf{w}^\top\mathbf{x}_i + b) < 1. \end{cases}$$

Другими словами, максимальное значение $\max_{\boldsymbol{\alpha};\boldsymbol{\alpha}\geq 0} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})$ возможно только при соблюдении изначального условия.

Теперь рассмотрим случай, когда $y_i(\mathbf{w}^\top\mathbf{x}_i + b) \geq 1$. Например,

$$y_i(\mathbf{w}^\top\mathbf{x}_i + b) = 3$$

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\mathbf{w}^\top\mathbf{w} - \sum_{i=1}^{N} \alpha_i(3-1) = \frac{1}{2}\mathbf{w}^\top\mathbf{w} - \sum_{i=1}^{N} 2\alpha_i$$

В силу того, что $\boldsymbol{\alpha}; \boldsymbol{\alpha} \geq 0$ всегда выполняется

$$-\sum_{i=1}^{N} 2\alpha_i \leq 0$$

Как следствие, при $y_i(\mathbf{w}^\top\mathbf{x}_i + b) \geq 1$ функция $\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})$ всегда достигает своего максимума в точке $\frac{1}{2}\mathbf{w}^\top\mathbf{w}$.

$$\max_{\boldsymbol{\alpha};\boldsymbol{\alpha}\geq 0} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\mathbf{w}^\top\mathbf{w}$$

Поскольку изначальной задачей была минимизация $\frac{1}{2}\mathbf{w}^\top\mathbf{w}$, то

$$\min_{\mathbf{w},b} \frac{1}{2}\mathbf{w}^\top\mathbf{w} = \min_{\mathbf{w},b} \left(\max_{\boldsymbol{\alpha};\boldsymbol{\alpha}\geq 0} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) \right)$$

Двойственная задача

Найдем и приравняем к нулю частные производные $\mathcal{L}$ относительно $\mathbf{w}$.

$$
\begin{aligned}
\frac{\partial \mathcal{L}}{\partial \mathbf{w}} &= \mathbf{w} - \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i = 0 \\
\mathbf{w} &= \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i
\end{aligned}
$$

Заметим, что вектор параметров $\mathbf{w}$ представляет собой линейную комбинацию векторов объектов.

Теперь найдем производные относительно b.

$$
\begin{aligned}
\frac{\partial \mathcal{L}}{\partial b} = -\sum_{i=1}^N \alpha_i y_i &= 0 \\
\sum_{i=1}^N \alpha_i y_i &= 0
\end{aligned}
$$

Подставим найденное значение $\mathbf{w}$ в $\mathcal{L}$.

$$
\begin{aligned}
\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = & \frac{1}{2}\mathbf{w}^\top \mathbf{w} - \sum_{i=1}^N \alpha_i [y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1] \\
= & \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \\
& - \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j + b \sum_{i=1}^N \alpha_i y_i + \sum_{i=1}^N \alpha_i \\
= & \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j + b \sum_{i=1}^N \alpha_i y_i
\end{aligned}
$$

Поскольку $\sum_{i=1}^N \alpha_i y_i = 0$,

$$
\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j
$$

Таким образом, мы получили эквивалентную или **двойственную задачу** (dual problem) оптимизации.

$$
\max_{\boldsymbol{\alpha}; \boldsymbol{\alpha} \geq 0} \left(\min_{\mathbf{w}, b} \mathcal{L}(\mathbf{w}, b, \alpha_i) \right)
$$

При этом опять же,

$$
\min_{\mathbf{w}, b} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \begin{cases} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j & \sum_{i=1}^N \alpha_i y_i = 0 \\ -\infty & \sum_{i=1}^N \alpha_i y_i \neq 0 \end{cases}
$$

Как следствие,

$$\max_{\boldsymbol{\alpha};\boldsymbol{\alpha}\geq 0}\left(\sum_{i=1}^{N}\alpha_i-\frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j\right)$$

$$\min_{\boldsymbol{\alpha};\boldsymbol{\alpha}\geq 0}\left(-\sum_{i=1}^{N}\alpha_i+\frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j\right)$$

с ограничением $\sum_{i=1}^{N}\alpha_i y_i = 0$.

Условия Каруша — Куна — Таккера

Заметим, что основная и двойственная задачи связаны следующим неравенством (принцип слабой двойственности, weak duality):

$$d^* = \max_{\boldsymbol{\alpha};\boldsymbol{\alpha}\geq 0}\min_{\mathbf{w},b}\mathcal{L}(\mathbf{w},b,\boldsymbol{\alpha}) \leq \min_{\mathbf{w},b}\max_{\boldsymbol{\alpha};\boldsymbol{\alpha}\geq 0} = p^*$$

где d^* — оптимальное решение двойственной задачи, а p^* — оптимальное решение основной задачи.

Неравенство превращается в равенство (принцип сильной двойственности, strong duality)

$$d^* = \mathcal{L}(\mathbf{w}^*, b^*, \boldsymbol{\alpha}^*) = p^*$$

в случае, если выполняются **условия Каруша — Куна — Таккера** (Karush — Kuhn — Tucker conditions, KKT), расширяющие метод множителей Лагранжа на случай ограничений, заданных неравенством. Приведем эти условия.

Условие стационарности (stationary condition)

$$\frac{\partial\mathcal{L}}{\partial\mathbf{w}^*} = 0, \quad \frac{\partial\mathcal{L}}{\partial b^*} = 0 \tag{1}$$

Основное условие осуществимости (primal feasibility condition)

$$y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) - 1 \geq 0 \tag{2}$$

Двойственное условие осуществимости (dual feasibility condition)

$$\alpha_i^* \geq 0 \tag{3}$$

Несложно убедиться, что сформулированная выше задача оптимизации соответствует каждому из этих условий.

Еще одним условием является *условие дополняющей нежесткости* (complementary slackness condition)

$$\alpha_i^*[y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) - 1] = 0, \tag{4}$$

которое можно записать и так:

$$\begin{aligned} &\alpha_i^* > 0 &&\implies && y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) - 1 = 0 \\ &y_i(\mathbf{w}^{*\top} \cdot \mathbf{x}_i + b^*) - 1 > 0 &&\implies && \alpha_i^* = 0 \end{aligned}$$

Другими словами, если $\alpha_i^* \neq 0$, то неравенство $y_i(\mathbf{x}_i^\top \mathbf{w} + b) - 1 \geq 0$ обращается в равенство и $\mathbf{x}_i$ лежат на H_- и H_+. Таким образом, параметр $\mathbf{w}$ зависит лишь от некоторых (опорных) векторов обучающей выборки.

Квадратичное программирование

Задача сводится к оптимизации квадратичного функционала

$$\max_{\boldsymbol{\alpha}} \left(\sum_{i=1}^N \alpha_i - \frac{1}{2} \boldsymbol{\alpha}^\top H \boldsymbol{\alpha} \right),$$

где $H_{ij} = y_i y_j \mathbf{x}_i^\top \mathbf{x}_j = \mathbf{y}\mathbf{y}^\top \odot \mathbf{x}_i^\top \mathbf{x}_j$ (и $\odot$ означает произведение Адамара) с ограничениями

$$\begin{aligned} &\alpha_i \geq 0 \\ &\textstyle\sum_{i=1}^N \alpha_i y_i = 0 \end{aligned}$$

Заметим, что поскольку матрица H и накладываемые ограничения неотрицательно определены (positive semidefinite), то речь идет об оптимизации выпуклого функционала с единственным решением.

При минимизации, векторизовав $\sum_{i=1}^{N} \alpha_i = \mathbf{1}^\top \boldsymbol{\alpha}$ и $\sum_{i=1}^{N} \alpha_i y_i = \mathbf{y}^\top \boldsymbol{\alpha}$, получим

$$\min_{\boldsymbol{\alpha}} \left(\frac{1}{2} \boldsymbol{\alpha}^\top H \boldsymbol{\alpha} - \mathbf{1}^\top \boldsymbol{\alpha} \right),$$

$$-\boldsymbol{\alpha} \leq \mathbf{0}$$
$$\mathbf{y}^\top \boldsymbol{\alpha} = 0$$

Линейное ядро

Функция $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$ называется **линейным ядром** (linear kernel function) алгоритма SVM.

В случае матрицы признаков $X_{n \times d}$ значения ядерной матрицы $K_{n \times n}$ (kernel matrix), также называемой матрицей Грама (Gram matrix), будут представлять собой скалярные произведения всех возможных пар n наблюдений X.

В коде линейное ядро можно реализовать следующим образом.

```
K = np.zeros((n_samples, n_samples))
for i in range(n_samples):
  for j in range(n_samples):
    K[i,j] = np.dot(X[i], X[j])
```

Матрица H, таким образом, будет иметь вид $H_{ij} = \mathbf{y}\mathbf{y}^\top \odot K$.

```
H = np.outer(y,y) * K
```

Одновременно матрицу H можно найти и так

$$H = (\mathbf{y} \odot X) \cdot (\mathbf{y} \odot X)^\top$$

```
H = np.dot(y*X, (y*X).T)
```

с тем неудобством, что вектор `y` с обычной размерностью `(n,)` придется преобразовывать в `(n,1)`.

Параметры и прогноз

Параметры $\mathbf{w}^*$ и b^* можно выразить через

$$\mathbf{w}^* = \sum_{s=1}^{S} \alpha_i y_i \mathbf{x}_i = (\mathbf{y}\boldsymbol{\alpha})^\top \mathbf{x}_i$$

$$b^* = \frac{1}{S} \sum_{i=1}^{S} (y_i - \sum_{i=1}^{S} \alpha_i y_i \mathbf{x}_i^\top \mathbf{x}_j) = \frac{1}{S} \sum_{i=1}^{S} \mathbf{y} - \mathbf{w}^{*\top} \mathbf{x}_j$$

При этом опять же учитываются только опорные векторы, для которых $\alpha_i > 0, i = 1, \ldots, S$.

Примечание. Заметим, что параметр b^* можно найти на основе одного (произвольного) опорного вектора $\mathbf{x}_s$ и соответствующей метки y_s

$$b^* = y_s - \sum_{i=1}^{S} \alpha_i y_i \mathbf{x}_i^\top \mathbf{x}_s$$

Среднее арифметическое (или медиана) берутся для обеспечения вычислительной устойчивости алгоритма.

Правило для классификации нового объекта $\mathbf{x}'$ можно сформулировать следующим образом.

$$\hat{y} = sgn(\mathbf{w}^{*\top} \mathbf{x}' + b^*) = sgn\left(\sum_{i=1}^{S} \alpha_i y_i \mathbf{x}_i^\top \mathbf{x}' + b^*\right),$$

где $\hat{y} = \{-1, +1\}$ и sgn — сигнум-функция.

Код на Питоне

Библиотека CVXOPT

Воспользуемся библиотекой CVXOPT⧉ для того, чтобы решить задачу квадратичного программирования.

$$\min_x \frac{1}{2} x^\top P x + q^\top x$$

$$Gx \preccurlyeq h$$
$$Ax = b$$

Приведем задачу к требуемому **функцией cvxopt.solvers.qp()** формату. В частности,

cvxopt.solvers.qp(P, q, G, h, A, b),

где

- P — матрица H размерностью $n \times n$;
- q — вектор $-\mathbf{1}_{n\times 1}$;
- G — диагональная матрица $n \times n$ со значениями -1 на главной диагонали;
- h — нулевой вектор $\mathbf{0}_{n\times 1}$;
- A — вектор меток $\mathbf{y}_{n\times 1}$;
- b — ноль (скаляр).

Таким образом, поставленная задача полностью совпадает с приведенной выше задачей минимизации функционала относительно $\boldsymbol{\alpha}$.

Класс HardSVM

Создадим алгоритм SVM с жестким зазором.

Откроем блокнот к занятию

```
class HardSVM:

  def __init__(self):
    self.support_alphas = None
    self.support_vectors = None
```

```
    self.support_vector_labels = None
    self.W = None
    self.intercept = None

  def fit(self, X, y):
    n_samples, n_features = np.shape(X)

    # вычислим ядерную матрицу
    K = np.zeros((n_samples, n_samples))
    for i in range(n_samples):
      for j in range(n_samples):
        K[i,j] = np.dot(X[i], X[j])

    # минимизируем квадратичный функционал
    P = cvxopt.matrix(np.outer(y,y) * K)
    q = cvxopt.matrix(np.ones(n_samples) * -1)
    # tc='d' переводит даннные в числа с плавающей точкой
    A = cvxopt.matrix(y, (1, n_samples), tc='d')
    b = cvxopt.matrix(0.0)
    G = cvxopt.matrix(np.identity(n_samples) * -1)
    h = cvxopt.matrix(np.zeros(n_samples))

    solution = cvxopt.solvers.qp(P, q, G, h, A, b)
    alphas = np.ravel(solution['x'])

    # найдем ненулевые множители Лагранжа,
    idx = alphas > 1e-5
    self.support_alphas = alphas[idx]
    # опорные векторы и соответствующие метки
    self.support_vectors = X[idx]
    self.support_vector_labels = y[idx]

    self.W = np.dot((self.support_vector_labels * self.support_alpl
    self.intercept = np.mean(self.support_vector_labels - np.dot(s

    return self

  def predict(self, X):

    return np.sign(np.dot(X, self.W) + self.intercept)
```

Линейно разделимые данные

Подготовим набор линейно разделимых данных.

```
def get_separable_data():
```

```
X_train = np.array([[-3.5, -1],[-3, 0], [-3, 1], [-2.7, -1.3], [-
                    [-1, -2.5], [0, -3], [0, 1], [0, 2.5], [1, 2]
                    [2.1, 0.2], [2.3, 1], [2.8, 1.8], [2.2, 2.8]]
y_train = np.array([-1, -1, -1, -1, -1, -1, -1, -1, 1, 1, 1, 1, 1
X_test = np.array([[0, 0], [1, -3]])
y_test = np.array([1, -1])

return X_train, y_train, X_test, y_test
```

Уравнение решающей границы

Кроме этого, создадим функцию, которая будет выводить данные, а также решающую границу H и границы коридора H_- и H_+.

Для двух признаков x_1 и x_2 получим

$$\mathbf{w}^\top \mathbf{x}_i + b = 0 \qquad \Longrightarrow \qquad \begin{bmatrix} w_1 & w_2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + b = 0$$

$$w_1 x_1 + w_2 x_2 + b = 0$$

$$w_2 x_2 = -w_1 x_1 - b$$

$$x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2},$$

где $-\frac{w_1}{w_2}$ — наклон, а $-\frac{b}{w_2}$ — сдвиг H. Кроме этого, мы знаем, что половина ширины коридора (margin) равна

$$\text{margin} = \frac{1}{||\mathbf{w}||}$$

При этом, поскольку $\mathbf{w}$ ортогонален H, то для нахождения H_- и H_+ необходимо нормализовать $\mathbf{w}$,

$$\hat{\mathbf{w}} = \frac{\mathbf{w}}{||\mathbf{w}||},$$

а затем сдвинуть H вверх и вниз на $\text{margin} \cdot \hat{\mathbf{w}}$

$$H_- = H - \text{margin} \cdot \hat{\mathbf{w}}$$

$$H_+ = H + \text{margin} \cdot \hat{\mathbf{w}}$$

Объявим функцию plot_data().

```
def plot_data(X, y, title, w=None, b=None):
  plt.figure(figsize=(5, 5))
  plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train,
              cmap=colors.ListedColormap(clrs))

  x_min, x_max, y_min, y_max = -4, 4, -4, 4
  xx = np.linspace(x_min, x_max)

  hyperplane_slope = -w[0]/w[1]
  hyperplane_intercept = -b/w[1]
  yy = hyperplane_slope*xx + hyperplane_intercept
  margin = 1 / np.sqrt(np.sum(w**2))
  w_hat = w / np.linalg.norm(w)
  hyperplane = np.array(list(zip(xx, yy)))
  hyperplane_plus = hyperplane + margin * w_hat
  hyperplane_minus = hyperplane - margin * w_hat

  plt.plot(hyperplane[:, 0], hyperplane[:, 1], 'k-')
  plt.plot(hyperplane_plus[:, 0], hyperplane_plus[:, 1], 'g--')
  plt.plot(hyperplane_minus[:, 0], hyperplane_minus[:, 1], 'g--')

  plt.xlim(x_min, x_max)
  plt.ylim(y_min, y_max)
  plt.title(title)
  plt.xlabel('x1')
  plt.ylabel('x2')
  plt.show()
```

Обучение и прогноз модели

Создадим датасеты.

```
X_train, y_train, X_test, y_test = get_separable_data()
X_train.shape, y_train.shape
```

```
((16, 2), (16,))
```

Обучим модель и сделаем прогноз.

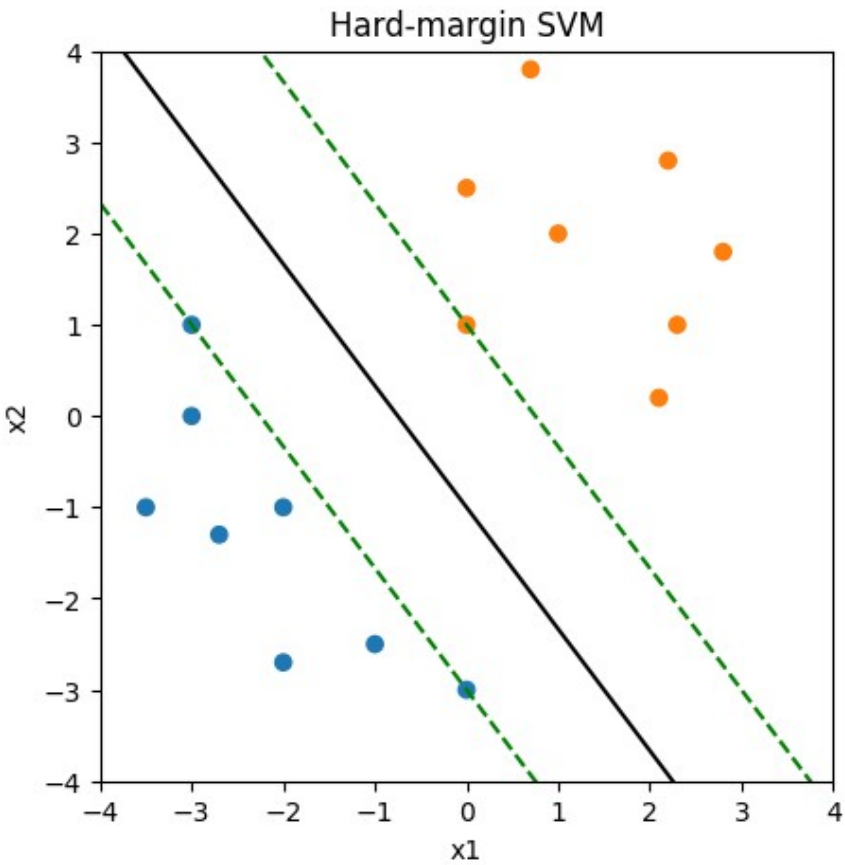
```
hard_svm = HardSVM()
hard_svm.fit(X_train, y_train).predict(X_test)
```

```
array([ 1., -1.])
```

Выведем опорные векторы.

```
hard_svm.support_vectors
```

```
array([[-3.,  1.],
       [ 0., -3.],
       [ 0.,  1.]])
```

Посмотрим на данные и гиперплоскость.

```
plot_data(X_train, y_train, title='Hard-margin SVM',
          w=hard_svm.W, b=hard_svm.intercept)
```